React Native + Full-Stack Interview Preparation

100 Questions & Answers — Beginner Friendly

Vol. 1: Resume & Projects · Vol. 2: AI / Native / Mobile Deep Dive

Prepared for

Mohammed Rinshad M I

AI-Augmented React Native Engineer

Palakkad, Kerala · 2026

Table of Contents

Section 6 — Native, Push & Real-Time

Q37. What is a native module in React Native, and when do you need to build one?
Q38. How do push notifications work end-to-end with FCM and APNs?
Q39. What are background tasks in mobile apps, and what limitations should you know about?
Q40. How do you implement biometric authentication in React Native?
Q41. What's the difference between WebSockets and HTTP polling, and when would you use each?

Section 7 — AI / LLM on Mobile & Backend

Q42. What is the Claude API, and how do you call it from a mobile app safely?
Q43. What is the Vercel AI SDK and how does it help with streaming?
Q44. What's the difference between REST APIs and WebSockets, and when do you use each?
Q45. What is JWT and how do you use it for authentication?
Q46. When would you use SQLite vs MongoDB vs Redis?

Section 8 — Debugging, Teamwork & Communication

Q47. Walk me through how you'd debug an API call that's failing in a React Native app.
Q48. Tell me about a time you handled a production crash. What was your process?
Q49. How do you communicate a technical decision to a non-technical stakeholder?
Q50. How do you stay up to date with new AI tools and React Native changes?

Section 9 — AI Streaming, Chat & Cost Optimization

Q51. How does Server-Sent Events (SSE) actually work, and why is it preferred for LLM streaming?
Q52. How do you handle a streaming AI connection that drops mid-response?
Q53. How would you implement a 'Stop' button while a Claude or OpenAI response is streaming?
Q54. What is prompt caching and how does it save you money on the Claude API?
Q55. How do you manage the context window when a chat conversation gets very long?
Q56. How do you parse partial JSON when an LLM streams a tool call back to you?
Q57. What's the difference between system, user, and assistant messages in a chat API?
Q58. How do you render markdown smoothly while text is still streaming in?
Q59. How would you estimate the cost of an AI chat feature before shipping it?
Q60. What's a typical latency budget for a 'feels real-time' AI chat on mobile?

Section 10 — Whisper, Voice AI & Audio Pipeline

Q61. What audio format should you send to Whisper, and why does it matter?
Q62. What sample rate and channel count does Whisper actually want?
Q63. How do you keep Whisper transcription latency low for short utterances?
Q64. What's the difference between Silero VAD and WebRTC VAD?
Q65. What is AVAudioSession on iOS and why does it matter for voice apps?
Q66. How do you handle echo cancellation when TTS and mic are both active?
Q67. How do you pick the right TTS voice — what trade-offs matter?
Q68. What's a typical end-to-end latency budget for a voice AI turn?

Section 11 — Tool Calling, Agents & Safety

Q69. How do you define a tool schema for Claude or OpenAI tool calling?
Q70. What happens if the LLM picks the wrong tool or calls it with invalid arguments?
Q71. How do you build a multi-step agent loop without it running forever?
Q72. How would you handle a dangerous tool — like 'delete account' — safely in an AI agent?
Q73. What's the difference between zero-shot, few-shot, and ReAct prompting?

Section 12 — Native Modules: Bridge, JSI & TurboModules

Q74. What is the React Native bridge, and why is it being replaced?
Q75. What is JSI and how is it different from the old bridge?
Q76. What are TurboModules and what problems do they solve?
Q77. What is Fabric in the React Native new architecture?
Q78. How do you expose a Kotlin function to JavaScript as a Promise?
Q79. How do you bridge Swift async/await to a JavaScript Promise?
Q80. How do you emit events from native code back to JavaScript?
Q81. What is a ViewManager and when would you need one?
Q82. What is autolinking in React Native and what does it do for you?
Q83. How would you debug a native module that’s crashing on Android?
Q84. How do you avoid memory leaks in a native module?
Q85. How do you keep a native module thread-safe?

Section 13 — Mobile Platform Integration

Q86. How do you set up FCM end-to-end for a React Native app?
Q87. What’s the difference between APNs certificates and auth keys, and which should you use?
Q88. What’s the difference between URL schemes, Universal Links, and App Links?
Q89. How do you request and check runtime permissions on iOS and Android?
Q90. How do In-App Purchases work and what’s the difference between StoreKit and Play Billing?
Q91. How do you handle the app foreground / background transitions?
Q92. How do you store a JWT or refresh token securely on a mobile device?
Q93. What is SafeAreaView and why do you need it?
Q94. How do you handle the iPhone notch, Dynamic Island, and Android cutouts?
Q95. How do you set up a custom splash screen and app icon for both platforms?

Section 14 — Real-time AI on Mobile & On-Device AI

Q96. How do you reconnect a WebSocket streaming AI session after a network drop?
Q97. What happens to a streaming AI request when the user backgrounds the app?
Q98. What are the battery implications of long-running AI sessions on mobile?
Q99. What is on-device AI, and which frameworks would you use — Core ML, TFLite, or ONNX?
Q100. When should you pick on-device AI vs cloud AI?

How to Use This Guide

This guide contains **100 beginner-level interview questions** in two volumes. **Volume 1 (Q1–Q50)** is curated from your resume — your role at **Infinite Open Source Solution LLP**, your three flagship projects, and your declared stack. **Volume 2 (Q51–Q100)** is a focused deep dive into **AI integration, native modules, and mobile platform integration** — the areas you work in every day. Each question follows the same structure: the question, **why interviewers ask it**, a **strong candidate answer** written in natural conversational language, and a **pro tip** to use in the room.

Read aloud, not silently. The goal isn't memorization — it's **making the answer feel like yours**. Replace examples with your own where you have stronger stories. Bold phrases in the answers are the keywords interviewers anchor to — say them clearly.

■ **Interview Question**	The exact question you'll likely hear.
■ **Why Interviewers Ask**	What the interviewer is really probing for.
■ **Strong Candidate Answer**	Natural, confident response — speak it like a conversation.
■ **Pro Tip**	One practical move that lifts an average answer to a strong one.

1 Section 1 — Experience at Infinite Open Source Solution LLP

Questions drawn from your Full Stack Developer role: TypeScript component library, AI features, JS→TS migration, release pipeline, native modules, billing.

QUESTION #1

■ Q1. Can you walk me through your current role at Infinite Open Source Solution LLP?

■ Why Interviewers Ask This

Interviewers want a clear, structured summary of your day-to-day work and the breadth of your responsibilities.

■ Strong Candidate Answer

Sure! I joined **Infinite Open Source Solution LLP** in November 2023 as a **Full Stack Developer**. My main focus is building **React Native** apps with a **TypeScript + Node.js** backend, and I've shipped **20+ web and mobile apps** across e-commerce, IoT, and AI domains. On a typical day I'm writing TypeScript on both client and server, integrating **REST** and **WebSocket** APIs, building **native modules in Kotlin** when a feature isn't available in JS, and pushing builds through **EAS** to the App Store and Play Store. I also own the AI-related work — streaming Claude and OpenAI responses, Whisper voice input — and the billing layer with Stripe and PayPal.

■ Pro Tip

*Always end the answer with what you **own end-to-end** — interviewers love candidates who can say 'this is mine, I ship it.'*

QUESTION #2

■ Q2. How did you cut feature delivery time by 30%? Can you give an example?

■ Why Interviewers Ask This

Tests whether you can quantify impact and explain the reasoning behind a productivity improvement.

■ Strong Candidate Answer

Yes — the biggest lever was building a **shared TypeScript component library**. Earlier, every project was re-implementing the same UI primitives — buttons, modals, form inputs, list rows — slightly differently. I extracted them into a single package with consistent props and theming, so a new screen that used to take a day now takes a few hours. I also added **Storybook** for visual previews and strict **TypeScript types** so misuse fails at compile time. After we rolled it out across three projects, feature lead-time dropped roughly **30%**, mostly because designers and engineers stopped re-debating basic UI patterns.

■ Pro Tip

*When you give a number like '30%', be ready for a follow-up: **how did you measure it?** Have a one-line answer ready, like 'we tracked Jira lead-time before and after rollout.'*

QUESTION #3

■ Q3. What does a 'cross-platform TypeScript component library' actually contain, and why is it useful?

■ Why Interviewers Ask This

Checks your understanding of code reuse, design systems, and the value of shared abstractions.

■ Strong Candidate Answer

It's basically a **shared npm package** with primitives that work on both **React** and **React Native**. So you have **Button**, **Input**, **Modal**, **Card**, plus higher-level patterns like **FormRow** and **EmptyState**. The trick is the components expose the same **TypeScript props** everywhere, but internally they render **div / input** on web and **View / TextInput** on native. The value is consistency — one bug fix updates every app — and speed, because engineers stop re-styling the same elements per project. It also makes **design hand-off** much cleaner since designers can reference one library.

■ Pro Tip

*Mention **versioning strategy** if asked deeper — say you use semantic versioning so consuming apps can upgrade safely.*

QUESTION #4

■ Q4. What AI-powered mobile features have you actually shipped to production?

■ Why Interviewers Ask This

Confirms your AI claims are real and tests how well you can talk through implementation details.

■ Strong Candidate Answer

I've shipped **streaming Claude and OpenAI chat** directly inside React Native apps — where each token appears as it arrives, instead of waiting for the whole response. I've integrated **Whisper** for **voice-to-text**, so the user can speak and we transcribe in real time. And I've built **conversational AI flows** with **tool calling**, where the LLM can trigger native actions like 'set a reminder' or 'open settings.' All of it runs inside our React Native shell, talking to the model APIs over **HTTPS / WebSockets**, with a small Node.js proxy to keep API keys off the device.

■ Pro Tip

*Always mention the **API key proxy** — interviewers will check if you understand never to ship secrets in the mobile bundle.*

QUESTION #5

■ Q5. Why did you migrate a live codebase from JavaScript to TypeScript? What problem were you trying to solve?

■ Why Interviewers Ask This

Probes your reasoning, not just your tooling — checks if you understand the why behind a big refactor.

■ Strong Candidate Answer

We were hitting the same **runtime errors** over and over — undefined props, wrong shape from the API, typos in keys. They'd only show up in production because JavaScript doesn't catch them at build time. So I proposed moving to **TypeScript** to catch that whole class of bugs at **compile time**. We did it **incrementally** — renaming files .ts/.tsx one by one and starting with **strict: false**, then tightening the config as we converted modules. Within a few months, the recurring 'cannot read property of undefined' bugs basically disappeared, and onboarding new devs got faster because types act as inline documentation.

■ Pro Tip

*Frame TypeScript as a **safety net + documentation**, not as 'extra work.' That mindset is what senior interviewers want to hear.*

QUESTION #6

■ Q6. How did you handle the JS-to-TypeScript migration without breaking the live app?

■ Why Interviewers Ask This

Checks for safe-rollout instincts — important for any production change.

■ Strong Candidate Answer

Carefully and gradually. First I added a TypeScript config with **allowJs: true** so .js and .ts files could coexist. Then I converted **leaf files first** — utility functions and small components — because they had the fewest dependents. Each PR was small and code-reviewed. We kept the **existing tests** green and ran the app on a dev device daily to catch regressions. For API responses I created shared **type definitions** aligned with the backend. Anything risky went behind a **feature flag**. The whole thing took a few months, but we never had to do a 'big bang' release — production stayed stable the whole time.

■ Pro Tip

*Use the phrase **'incremental migration'** — interviewers immediately associate it with safe, mature engineering practice.*

QUESTION #7

■ Q7. You mentioned 'owning the release pipeline.' What does that include?

■ Why Interviewers Ask This

Verifies whether you actually shipped apps or just wrote code others released.

■ Strong Candidate Answer

It means I handle the app end-to-end from build to live store. On **iOS** that's managing the **Apple Developer account**, generating **signing certificates** and **provisioning profiles**, archiving with **Xcode**, and uploading via **EAS Submit** or Transporter. On **Android** it's the **keystore**, the **signed AAB**, and uploads through the Play Console. I also write the release notes, set up **staged rollouts** — usually 10% then 50% then 100% — and watch **Sentry** for crash spikes. If something breaks, I can roll back. I've done this independently for **5+ apps**.

■ Pro Tip

*Mention **staged rollout** specifically — it's one of those small details that shows you've actually published apps.*

QUESTION #8

■ Q8. What native modules have you built in Kotlin, and why was a native module needed?

■ Why Interviewers Ask This

Tests your understanding of when JS isn't enough and how the bridge works.

■ Strong Candidate Answer

Three main ones. First, **background geolocation** — the JS-only libraries couldn't keep tracking after the app was killed, so I wrote a Kotlin service that uses **FusedLocationProviderClient** and a foreground notification. Second, **push notifications** with **FCM** — to handle data-only messages and deep-link routing into specific screens. Third, **biometric auth** using Android's **BiometricPrompt** for fingerprint and face unlock. In each case the JS side calls a method exposed through React Native's **native module bridge**, and the native code does the platform-specific work and returns a promise.

■ Pro Tip

*Be ready to draw the **JS** ↔ **bridge** ↔ **native** flow on a whiteboard — interviewers love when you visualize it.*

QUESTION #9

■ Q9. How did you process Stripe and PayPal subscriptions with zero reconciliation failures?

■ Why Interviewers Ask This

Checks if you understand payment flows, webhooks, and idempotency — common pitfalls for juniors.

■ Strong Candidate Answer

Three things made it work. First, every payment event came in through a **webhook**, and I always verified the **signature** using the Stripe and PayPal SDKs — so spoofed requests were rejected. Second, every charge had an **idempotency key** tied to the order ID, so if the webhook fired twice we didn't double-charge or double-credit. Third, I built a **reconciliation job** that runs nightly and compares our DB records to Stripe's API — if anything's off, it alerts. With those three layers, we've had **zero mismatches** in production.

■ Pro Tip

*Use the magic word **'idempotent'** — every backend interviewer is listening for it in payment questions.*

QUESTION #10

■ Q10. What is webhook verification, and why is it important?

■ Why Interviewers Ask This

Beginner-friendly security question — checks fundamentals of trusting external events.

■ Strong Candidate Answer

A **webhook** is just an HTTP request that an external service like Stripe sends to my server when something happens — say a payment succeeds. The problem is, anyone on the internet can hit that URL and pretend to be Stripe. **Webhook verification** means checking a **cryptographic signature** in the request header, computed with a **secret key** only Stripe and I share. If the signature doesn't match, I reject the request. Without this, an attacker could fake a 'payment succeeded' event and trigger us to ship a free product.

■ Pro Tip

Always tie security questions back to a concrete attack — interviewers want to see you think like an attacker.

2 Section 2 — Project: AI Life Assistant Super App

Voice-first AI assistant — streaming LLM chat, Whisper, TTS, tool calling, native audio module.

QUESTION #11

■ Q11. Walk me through your AI Life Assistant Super App at a high level.

■ Why Interviewers Ask This

Tests storytelling — can you explain a complex product in plain language?

■ Strong Candidate Answer

Absolutely. It's a **voice-first AI assistant** built in React Native. The user can tap a mic button and speak — we capture audio with a **custom native module**, transcribe it with **Whisper**, send it to **Claude or OpenAI** as a streaming chat, and then read the answer back using **TTS**. The assistant can also call **tools** like 'create a reminder' or 'fetch my calendar', and the message UI animates at **60fps** using **Reanimated**. We also support **offline-first SQLite** so the user's history persists, and we use **FCM/APNs** push so the agent can proactively nudge the user — like 'you said to remind you about the gym, here's your reminder.'

■ Pro Tip

*Start a project answer with **what the product does for the user**, not the tech stack. Tech comes second.*

QUESTION #12

■ Q12. What is streaming LLM chat, and why is it better than waiting for a full response?

■ Why Interviewers Ask This

Beginner-friendly AI concept that affects perceived performance.

■ Strong Candidate Answer

Streaming LLM chat means the model sends back **tokens one at a time** instead of returning the whole answer at once. So instead of staring at a spinner for 8 seconds, the user starts seeing words appear within half a second — just like how ChatGPT renders. Technically, we keep an open connection — usually **Server-Sent Events** or a WebSocket — and append each token to the message bubble as it arrives. The total time is similar, but the **perceived latency** is dramatically lower, and users feel the app is fast and alive.

■ Pro Tip

*Use the phrase **'perceived latency'** — it shows you think about UX, not just network performance.*

QUESTION #13

■ Q13. How does Whisper transcription work in your app?

■ Why Interviewers Ask This

Tests practical AI integration knowledge.

■ Strong Candidate Answer

Whisper is OpenAI's speech-to-text model. In the app, when the user taps the mic, our **native audio module** records a short audio clip — usually in WAV or M4A — and we send it to the Whisper API as a multipart upload. Whisper returns the transcribed text in a few hundred milliseconds, and we either show it as user input in the chat or feed it directly to the LLM as the next message. For longer recordings we **chunk the audio** so the user sees partial transcripts coming back, which keeps it feeling responsive.

■ Pro Tip

*Mention **chunking** for long audio — interviewers love when you anticipate the 'what about a 10-minute recording' edge case.*

QUESTION #14

■ Q14. What is TTS playback and how did you implement it?

■ Why Interviewers Ask This

Checks knowledge of audio output APIs and user experience.

■ Strong Candidate Answer

TTS stands for **text-to-speech** — it turns the LLM's text answer into spoken audio. I used a cloud TTS service (OpenAI TTS or ElevenLabs depending on the project) which returns an MP3 stream. On the React Native side, I pipe that audio into a player — **react-native-track-player** on top — and start playback as soon as the first chunk arrives. So the user hears the answer almost immediately. I also handle **interruption** — if the user starts speaking again, I stop playback and switch back to recording, which makes it feel like a real conversation.

■ Pro Tip

*Bring up **barge-in** — letting the user interrupt the AI mid-sentence. It's a small but advanced UX detail.*

QUESTION #15

■ Q15. What is 'tool calling' in the context of AI assistants?

■ Why Interviewers Ask This

Tests whether you understand modern LLM architecture, not just chat.

■ Strong Candidate Answer

Tool calling — sometimes called **function calling** — is where the LLM doesn't just reply with text but instead returns a **structured request** to run a function. For example, I tell Claude 'you have a tool called createReminder(title, time)', and when the user says 'remind me to call mom at 5', the model returns a JSON like **{ tool: 'createReminder', args: { title: 'call mom', time: '17:00' } }**. My app sees that, runs the actual native code to create the reminder, and sends the result back to the model so it can confirm to the user. It's how AI assistants **take real actions** instead of just chatting.

■ Pro Tip

Always close with 'and the result goes back to the model' — many candidates forget the loop and lose points.

QUESTION #16

■ Q16. Why did you build a custom native audio module instead of using an existing library?

■ Why Interviewers Ask This

Checks judgment about build vs buy.

■ Strong Candidate Answer

I tried libraries first — **expo-av** and **react-native-audio-recorder-player** — but they had two issues for our use case. First, **latency**: I needed sub-100ms mic capture for VAD to feel snappy, and the JS bridge added too much overhead. Second, I needed **Voice Activity Detection** on raw audio frames, which the libraries didn't expose. So I wrote a thin native module in **Swift** and **Kotlin** using **AVAudioEngine** on iOS and **AudioRecord** on Android, did the VAD natively, and only sent the resulting transcripts up to JS. It was about **2x faster** and used less battery.

■ Pro Tip

*Always justify a custom native module with a **concrete limitation** of the library — never 'just because.'*

QUESTION #17

■ Q17. What is VAD — Voice Activity Detection — and how is it useful?

■ Why Interviewers Ask This

Beginner-friendly question on a buzzword you used.

■ Strong Candidate Answer

VAD is a small algorithm that listens to incoming audio and tells you 'someone is speaking now' versus 'silence.' It’s useful because we don’t want to send **silence** or background noise to Whisper — that wastes API calls and money, and adds latency. With VAD, we automatically start recording when the user begins talking and stop a moment after they finish. So the user doesn’t even need to press a button — they just talk and the assistant responds. It’s what makes voice AI feel natural.

■ Pro Tip

*End with the UX win — **'no button needed'** — because the interviewer remembers the user impact more than the algorithm.*

3 Section 3 — Project: Cloud-Native IoT Analytics Dashboard

Live device telemetry over WebSockets, virtualized charts, offline sync, push alerts, Sentry tracing.

QUESTION #18

■ Q18. Tell me about your Cloud-Native IoT Analytics Dashboard project.

■ Why Interviewers Ask This

Lets you frame a real project naturally.

■ Strong Candidate Answer

It's a **mobile companion app** for a cloud IoT platform. Field engineers use it to monitor live device telemetry — temperature, voltage, signal strength — coming in from thousands of sensors. The data **streams over WebSockets** in real time into **virtualized charts**, so even on a low-end Android phone the UI stays smooth. They can **acknowledge alerts**, and if it's critical, the system **pages the on-call engineer** via push notification. When the phone goes offline — like in a basement or rural site — we **cache to SQLite** and sync back when the network returns. We also use **Sentry** for crash tracing.

■ Pro Tip

*Always mention **'low-end device'** — interviewers want to know you think about real-world hardware, not just iPhones.*

QUESTION #19

■ Q19. Why did you use WebSockets for streaming device telemetry instead of REST polling?

■ Why Interviewers Ask This

Tests understanding of real-time communication trade-offs.

■ Strong Candidate Answer

Polling REST endpoints every second works in theory but it's wasteful and slow. The phone has to keep opening new HTTPS connections, the server fires up handlers for empty responses, and the user still sees a 1-second lag. **WebSockets** open **one persistent connection**, and the server pushes data as it arrives. For IoT, where readings come every couple hundred milliseconds, that's a much better fit — lower battery use, lower latency, and the server can also push **alerts** without us asking.

■ Pro Tip

*Always mention **battery life** when comparing WebSockets vs polling — it's the answer most candidates miss.*

QUESTION #20

■ Q20. What are virtualized charts and why did you need them?

■ Why Interviewers Ask This

Beginner-friendly performance question.

■ Strong Candidate Answer

A virtualized chart only **renders the data points that are currently visible** on the screen, not the whole dataset. So if a device sends 10,000 readings over a day, we don't draw 10,000 dots — we draw the few hundred visible at the current zoom level. As the user scrolls or zooms, we swap them in and out. Without virtualization, the chart would freeze the UI thread on low-end devices and burn battery. I used **react-native-skia** and **victory-native** patterns depending on the chart type.

■ Pro Tip

*Mention **'UI thread blocking'** — it's the technical term that makes the answer sound senior.*

QUESTION #21

■ Q21. How does offline sync with SQLite plus backoff retries actually work?

■ Why Interviewers Ask This

Tests real-world offline strategy.

■ Strong Candidate Answer

When the device is online, every action — like acknowledging an alert — goes straight to the API. But I also write it to a local **SQLite** table called **pending_actions**. If the API call fails or the device is offline, the action stays in that table. A background task tries to flush the queue periodically with **exponential backoff** — first after 2 seconds, then 4, then 8, up to a max. When the network returns, it drains the queue in order. The user's actions never get lost, even if they spent two hours in a basement.

■ Pro Tip

*Always say **'queue draining in order'** — losing event order is a classic bug interviewers probe for.*

QUESTION #22

■ Q22. What's the difference between FCM and APNs?

■ Why Interviewers Ask This

Beginner-friendly push notification fundamentals.

■ Strong Candidate Answer

Both are **push notification services**, just for different platforms. **FCM — Firebase Cloud Messaging** — is Google's service for Android. **APNs — Apple Push Notification service** — is Apple's for iOS. From the server side, you send a payload — title, body, data — to either Google's or Apple's servers, and they deliver it to the device. The device **token** identifies which phone to deliver to, and it's unique per app install. I usually wrap both behind a single Node.js service so the app team only thinks about 'send notification', not which platform.

■ Pro Tip

*Mention the **device token** explicitly — interviewers want to know you've actually wired one up.*

QUESTION #23

■ Q23. What is Sentry, and how did it help you reach MTTR under 1 hour?

■ Why Interviewers Ask This

Checks knowledge of production observability.

■ Strong Candidate Answer

Sentry is a service that captures **crashes and errors** from your app in real time — with the full stack trace, the device model, OS version, and even the user actions that led up to it. So when a crash happens in production, I get a Slack alert within seconds, click into the error, and I already have everything I need to reproduce it. Combined with **source maps**, I can see the exact line in TypeScript — not minified JS. That's how we kept **MTTR — mean time to recover — under one hour**. Without Sentry, we'd be guessing from app store reviews.

■ Pro Tip

*MTTR stands for **Mean Time To Recover** — always say the expansion in your answer in case the interviewer doesn't know it.*

QUESTION #24

■ Q24. What does 'on-call paging' mean for a mobile app?

■ Why Interviewers Ask This

Beginner-friendly clarification on a domain term.

■ Strong Candidate Answer

On-call paging is when the system **wakes someone up** because a critical issue needs immediate attention. In our IoT dashboard, if a device fires a critical alert — say a sensor reading goes dangerously high — we send a **high-priority push notification** to the on-call engineer's phone. On iOS we use **critical alerts** which bypass silent mode, and on Android we send it as a **high-priority FCM message** that wakes the device. If they don't acknowledge within a few minutes, the system pages the next person in the rotation. It's basically PagerDuty inside our app.

■ Pro Tip

*Mention **'bypass silent mode'** for critical alerts — it shows you understand iOS notification entitlements at a deeper level.*

4 Section 4 — Project: AI-Powered Real-Time Collaboration Platform

CRDT collaboration with Yjs, WebRTC voice notes, inline streaming AI on document selections.

QUESTION #25

■ Q25. Can you explain your Real-Time Collaboration Platform project?

■ Why Interviewers Ask This

Lets you frame a complex project in your own words.

■ Strong Candidate Answer

Sure! It's a React Native app where multiple users edit the same document together — like Google Docs on mobile. Each user sees the others' cursors and selections live, with **presence avatars** and **typing indicators**. The magic underneath is **Yjs**, which is a **CRDT** library that lets edits merge automatically without conflicts — even when users are offline and reconnect later. We also added **WebRTC voice notes** so they can drop a short voice message, and **inline AI** where you can highlight a paragraph and ask OpenAI to summarize it, with the response streaming in at 60fps using Reanimated.

■ Pro Tip

*When describing collab products, always say **'like Google Docs / Figma / Notion'** — it instantly anchors the interviewer.*

QUESTION #26

■ Q26. What is a CRDT and why did you choose Yjs?

■ Why Interviewers Ask This

Beginner-friendly version of an advanced concept.

■ Strong Candidate Answer

CRDT stands for **Conflict-Free Replicated Data Type**. The idea is that two users can edit the same document **at the same time**, even offline, and when they sync, the changes merge automatically — no 'conflict, choose one' popup. It works because every edit carries a unique ID and a logical timestamp, so the algorithm can deterministically order them. **Yjs** is one of the best implementations — small bundle, mobile-friendly, has React bindings, and works over **WebSockets** or **WebRTC**. We chose it because it's production-proven — Linear and JupyterLab use it — and it just works out of the box.

■ Pro Tip

*Use the magic phrase **'conflict-free merge'** — that's the value proposition and what the interviewer wants to hear.*

QUESTION #27

■ Q27. How do live presence and typing indicators work in a collaborative app?

■ Why Interviewers Ask This

Beginner-friendly real-time UX question.

■ Strong Candidate Answer

Presence is basically **heartbeat messages**. Each user sends a small WebSocket message every few seconds saying 'I'm here, my cursor is at position X.' The server broadcasts that to other users, who render the avatar and cursor in real time. If a user goes silent for 30 seconds, we mark them as away. Typing indicators are the same idea — on a **debounced** keystroke, send 'I'm typing'; after 2 seconds of silence, send 'I stopped.' It's lightweight, and the **debounce** stops us from spamming the server every keypress.

■ Pro Tip

*Mention **debounce** — it's the small detail that prevents you from looking like you'd DDoS your own server.*

QUESTION #28

■ Q28. What is WebRTC, and how did you use it for voice notes?

■ Why Interviewers Ask This

Tests understanding of peer-to-peer media.

■ Strong Candidate Answer

WebRTC is a browser and mobile API for sending **audio, video, and data peer-to-peer**, with very low latency. For voice notes I didn't actually need real-time calling — I just used WebRTC's **audio capture** on the device to record, encoded it as Opus, and uploaded it to S3 with a shareable link. But the same API also lets us do **live voice calls** later by setting up a peer connection through a signaling server. The big win of WebRTC is the audio codec and echo cancellation are battle-tested by Google and built right in.

■ Pro Tip

*If asked about scaling WebRTC calls to many users, mention **SFU — Selective Forwarding Unit** — that shows you've thought past 1-on-1 calls.*

QUESTION #29

■ Q29. What is 'inline AI' and how did you implement it on document selections?

■ Why Interviewers Ask This

Beginner-friendly question on a feature you built.

■ Strong Candidate Answer

Inline AI is when the user **highlights text** inside the document and a small toolbar pops up — 'summarize', 'rewrite', 'translate.' When they tap one, we send the selected text plus the action prompt to **OpenAI** as a streaming request, and the response writes itself into the document **token by token**. So the user sees the AI typing right where they were working. The trick is using **Reanimated** to keep the cursor and surrounding text stable during the insert — otherwise it feels jumpy.

■ Pro Tip

*Mention **'no modal popup, no context switch'** — that's the UX value of inline AI vs a separate chat window.*

QUESTION #30

■ Q30. How do streaming OpenAI completions feel different to users compared to regular API calls?

■ Why Interviewers Ask This

Tests UX intuition.

■ Strong Candidate Answer

A regular API call shows a spinner for 5 to 10 seconds and then dumps the whole answer at once — it feels slow and lifeless. A **streaming completion** starts showing words within 200 to 500 milliseconds — the user sees the AI 'thinking out loud.' Even if total time is the same, the **perceived speed** is dramatically better. It also gives the user a chance to read along, stop the request early if they see it going off track, and feel more engaged. From an engineering side, it means we have to handle **partial responses** and the **cancel button** properly.

■ Pro Tip

*Always mention the **cancel button** — interviewers love when you bring up user agency, not just rendering speed.*

QUESTION #31

■ Q31. How did you achieve 60fps animations with Reanimated?

■ Why Interviewers Ask This

Tests mobile performance knowledge.

■ Strong Candidate Answer

Reanimated runs animations on the **UI thread** using its own native worklet runtime, instead of on the JS thread. That's the key — the regular React Native Animated API can stutter when the JS thread is busy parsing API responses, but Reanimated bypasses that entirely. So even while we're streaming tokens from OpenAI and re-rendering the document, the animation stays smooth at **60fps**. I also avoided animating layout properties — sticking to **transform** and **opacity** — and used **useSharedValue** and **useAnimatedStyle** properly.

■ Pro Tip

*Mention **'transform and opacity only'** — it's the universal rule for smooth animations on any platform.*

5 Section 5 — Languages, Frontend & State Management

TypeScript, Expo / EAS, Redux Toolkit, Zustand, React Query, Reanimated, Gesture Handler.

QUESTION #32

■ Q32. What's the difference between JavaScript and TypeScript, and when would you prefer one over the other?

■ Why Interviewers Ask This

Beginner-level fundamental, asked in almost every TS interview.

■ Strong Candidate Answer

JavaScript is the original language — it's flexible and runs anywhere, but it has no **type checking**, so bugs like passing a string where a number was expected only show up at runtime. **TypeScript** is a superset that adds **static types** on top, and a compiler that catches those mistakes **before** the code runs. For a **quick prototype or script**, plain JavaScript is fine — less ceremony. For anything **shipped to production or worked on by a team**, I always go TypeScript — the types act as inline documentation, refactoring becomes safe, and the editor autocomplete is much better.

■ Pro Tip

*Always link TypeScript to **team scale** — the bigger the team, the bigger the win.*

QUESTION #33

■ Q33. When would you use Redux Toolkit vs Zustand vs React Query?

■ Why Interviewers Ask This

Tests practical state management judgment.

■ Strong Candidate Answer

Each solves a different problem. **React Query** is for **server state** — API data, caching, refetching, loading states — that's 70% of what most apps need, so I start there. **Zustand** is for **simple client state** — a small global store with no boilerplate, perfect for UI state like a sidebar toggle or theme. **Redux Toolkit** I reach for when the app has **complex client state** with lots of interactions — multi-step wizards, offline queues, undo/redo — where I want the structure of actions, reducers, and middleware. Often I use React Query and Zustand together and skip Redux entirely.

■ Pro Tip

*Phrase it as **'server state vs client state'** — that distinction shows mature thinking.*

QUESTION #34

■ Q34. What is Expo and EAS, and why are they useful?

■ Why Interviewers Ask This

Beginner-friendly tooling question.

■ Strong Candidate Answer

Expo is a framework on top of React Native that gives you a smoother developer experience — pre-built native modules, easy access to camera, location, push, and so on, without writing native code. **EAS — Expo Application Services** — is their cloud build and deployment service. Instead of fighting with Xcode and Android Studio locally, you run **eas build** and it builds the iOS and Android binaries in the cloud. **eas submit** then uploads to the App Store and Play Store. For solo developers or small teams, it saves an enormous amount of time.

■ Pro Tip

*Mention **'no local Xcode required'** — that's the magic moment for anyone who's ever fought with code signing.*

QUESTION #35

■ Q35. What is React Query and what problem does it solve?

■ Why Interviewers Ask This

Beginner-friendly question about a common library.

■ Strong Candidate Answer

React Query — now called **TanStack Query** — manages **server data** in your React components. Before it, every developer wrote their own useEffect + useState + loading + error + retry logic for every API call. React Query gives you **useQuery** and **useMutation** hooks that handle caching, automatic refetching, stale-while-revalidate, retries on failure, and request deduplication — all out of the box. So instead of 30 lines of boilerplate per API call, you write three. And because it caches by query key, two components asking for the same data don't fire two requests.

■ Pro Tip

*Mention **'stale-while-revalidate'** — it's the buzzword that signals you understand caching deeply.*

QUESTION #36

■ Q36. What is Gesture Handler and why is it preferred over PanResponder?

■ Why Interviewers Ask This

Tests mobile gesture knowledge.

■ Strong Candidate Answer

react-native-gesture-handler is a library that runs gesture recognition **natively** on iOS and Android, instead of in the JS thread. The old built-in **PanResponder** ran in JavaScript, so if the JS thread was busy, your swipe would feel sticky or drop. Gesture Handler doesn't have that problem — gestures are smooth even under load. It also composes nicely with **Reanimated**, so you can do things like a swipe-to-dismiss with the position driven by gesture and the spring back driven by Reanimated, all on the UI thread. Smooth 60fps interactions.

■ Pro Tip

*Bundle **Gesture Handler + Reanimated** in your answer — they're the standard combo and interviewers expect both names.*

6 Section 6 — Native, Push & Real-Time

Native modules, FCM / APNs, background tasks, biometric auth, WebSockets vs polling.

QUESTION #37

■ Q37. What is a native module in React Native, and when do you need to build one?

■ Why Interviewers Ask This

Beginner-friendly question about bridging native code.

■ Strong Candidate Answer

A **native module** is platform-specific code — **Swift** or **Objective-C** on iOS, **Kotlin** or **Java** on Android — that you expose to JavaScript through the React Native bridge. You need one when JavaScript can't do something the platform requires: **background tasks that survive app kill**, **biometric prompts**, **Bluetooth**, **low-latency audio**, or any vendor SDK that only ships in native form. I usually try a community library first; if it doesn't exist or has limitations, I write a thin wrapper that exposes only the methods I need.

■ Pro Tip

*Always mention **'community library first'** — interviewers want to see judgment, not 'I write native code for everything.'*

QUESTION #38

■ Q38. How do push notifications work end-to-end with FCM and APNs?

■ Why Interviewers Ask This

Tests understanding of the full push flow.

■ Strong Candidate Answer

There are four pieces. First, the **device registers** with FCM or APNs at app launch and gets back a **device token**. Second, the app sends that token to **my backend** and saves it against the user. Third, when something happens — like an alert — my backend POSTs a notification payload to FCM or APNs with that token. Fourth, FCM or APNs delivers it to the device, the OS shows the banner, and tapping it deep-links the user into the right screen. If the app is killed, the OS still shows the banner. If it's in the foreground, my JS code handles it instead.

■ Pro Tip

*Always end with **'deep link to the right screen'** — many candidates forget the in-app routing piece.*

QUESTION #39

■ Q39. What are background tasks in mobile apps, and what limitations should you know about?

■ Why Interviewers Ask This

Practical question on a tricky area.

■ Strong Candidate Answer

Background tasks let your app keep doing work after the user leaves it — syncing data, tracking location, finishing an upload. The catch is both OSes are very strict to save battery. On **iOS**, you only get a **few seconds to a few minutes** via Background Tasks framework, and the system decides when to wake you. On **Android** it's more flexible with **WorkManager** and **foreground services**, but you must show a persistent notification or get killed. So the rule is: **do the minimum work**, persist state to disk, and assume you can be killed at any moment.

■ Pro Tip

*Use the phrase **'assume you can be killed'** — it shows you've been burned by it before, which is a sign of experience.*

QUESTION #40

■ Q40. How do you implement biometric authentication in React Native?

■ Why Interviewers Ask This

Beginner-friendly platform feature question.

■ Strong Candidate Answer

I use a library like **react-native-biometrics** or **expo-local-authentication**. The flow is simple: check whether the device supports biometrics, then call **authenticate** with a prompt message. On iOS, the OS shows the **Face ID or Touch ID** system sheet; on Android, it shows the **BiometricPrompt**. The library returns success or failure. The important detail is I never store the actual biometric — that stays in the **Secure Enclave** on iOS or the **TEE** on Android — the OS just tells me yes or no. I usually pair it with a **fallback to PIN**.

■ Pro Tip

*Mention **'Secure Enclave'** — it's the magic word that proves you understand the security model.*

QUESTION #41

■ Q41. What’s the difference between WebSockets and HTTP polling, and when would you use each?

■ Why Interviewers Ask This

Real-time fundamentals.

■ Strong Candidate Answer

HTTP polling is when the client repeatedly asks the server 'anything new?' every few seconds. It’s simple, works over normal HTTP, but it’s wasteful when nothing has changed, and there’s always a delay equal to the poll interval. **WebSockets** open a **persistent two-way connection** — the server pushes data as it happens, with no overhead per message. I use WebSockets for **chat, live cursors, IoT telemetry, notifications** — anything that needs **sub-second freshness**. I use polling only when updates are rare and infrastructure for WebSockets is too heavy.

■ Pro Tip

*Mention **'persistent two-way connection'** — that single phrase tells the interviewer you understand the model.*

7 Section 7 — AI / LLM on Mobile & Backend

Claude API, Vercel AI SDK, REST vs WebSockets, JWT, SQLite vs MongoDB vs Redis.

QUESTION #42

■ Q42. What is the Claude API, and how do you call it from a mobile app safely?

■ Why Interviewers Ask This

Tests AI integration and security awareness.

■ Strong Candidate Answer

The **Claude API** is Anthropic's REST API for talking to the Claude language model. You send a chat history with a system prompt and a list of messages, and it returns the model's reply — either as one response or as a **streaming SSE** connection. From a mobile app, I never call it directly — that would expose my API key to anyone who decompiles the app. Instead, I put a **Node.js proxy** in the middle: the mobile app authenticates with my backend, the backend holds the Anthropic key and forwards the request to Claude. That way the key stays secret and I can add **rate limiting** and **logging**.

■ Pro Tip

*The phrase **'never put API keys in the mobile bundle'** is the #1 security gotcha — always mention it.*

QUESTION #43

■ Q43. What is the Vercel AI SDK and how does it help with streaming?

■ Why Interviewers Ask This

Beginner-friendly question on a popular library.

■ Strong Candidate Answer

The **Vercel AI SDK** is a small library that wraps the messy parts of calling LLMs — Claude, OpenAI, Gemini — behind one consistent interface. It handles **streaming** over Server-Sent Events out of the box, manages the message format, supports **tool calling**, and on the React side gives you a **useChat** hook that auto-renders streaming responses into the UI. Without it, you write a lot of boilerplate around fetch, SSE parsing, and partial JSON. With it, you're shipping a streaming chat UI in about ten lines. I've used it both in Next.js backends and in React Native via a custom hook.

■ Pro Tip

*Mention the **useChat** hook by name — it's the signature feature interviewers look for.*

QUESTION #44

■ Q44. What's the difference between REST APIs and WebSockets, and when do you use each?

■ Why Interviewers Ask This

Beginner-friendly backend communication question.

■ Strong Candidate Answer

REST is a request-response model — client asks, server replies, connection closes. Great for **CRUD**: fetch a user, create a post, delete an item. It's simple, cacheable, and works everywhere. **WebSockets** are **bidirectional and persistent** — both sides can send messages whenever, with no per-message overhead. Great for **real-time**: chat, presence, live dashboards, streaming. In practice I use REST for most operations and **add WebSockets for the few real-time features**, so I'm not paying the complexity tax everywhere.

■ Pro Tip

*Always say **'REST for CRUD, WebSockets for real-time'** — it's the one-liner interviewers want to hear.*

QUESTION #45

■ Q45. What is JWT and how do you use it for authentication?

■ Why Interviewers Ask This

Beginner-friendly auth fundamentals.

■ Strong Candidate Answer

JWT stands for **JSON Web Token**. It's a small, signed string that contains the user's identity — like **{ userId: 123, role: 'admin' }** — plus an expiry. When the user logs in, my backend signs a JWT with a secret key and sends it back. The client stores it (in **Keychain** on iOS or **Keystore** on Android — not AsyncStorage), and includes it in the **Authorization: Bearer** header on every API request. My backend verifies the signature, and if valid, trusts the user ID inside. The big win is the backend doesn't need to look up the session in a database on every request.

■ Pro Tip

*Mention **Keychain / Keystore** instead of AsyncStorage — it's the security detail interviewers reward.*

QUESTION #46

■ Q46. When would you use SQLite vs MongoDB vs Redis?

■ Why Interviewers Ask This

Beginner-friendly data store question.

■ Strong Candidate Answer

Different jobs. **SQLite** is a tiny, file-based SQL database — I use it **on the device** for offline caches and local storage. **MongoDB** is a document database on the server — great when the data shape is flexible or nested, like user profiles, chat messages, or event logs. **Redis** is an **in-memory store** — blazing fast, used for **caching, session storage, rate limiting, and pub/sub**. A typical stack: MongoDB as the source of truth, Redis in front for hot reads and pub/sub, and SQLite on the mobile client for offline.

■ Pro Tip

*Always close with **'they're complementary, not competing'** — that nuance shows real production experience.*

8 Section 8 — Debugging, Teamwork & Communication

Real-world debugging stories, production crashes, stakeholder communication, learning AI tools.

QUESTION #47

■ Q47. Walk me through how you'd debug an API call that's failing in a React Native app.

■ Why Interviewers Ask This

Tests systematic debugging instincts.

■ Strong Candidate Answer

I always go layer by layer. First, I check the **network** — is the device online, can it reach the server at all? I use **Flipper** or **Reactotron** to watch the actual HTTP request and response. Second, I check the **request shape** — headers, body, auth token — to make sure I'm sending what the backend expects. Third, I look at the **backend logs** to see if the request even arrived and what the server saw. Fourth, if the response is correct but the UI is wrong, the bug is in my **state or rendering**. The mistake juniors make is jumping into code first; I always confirm the request and response first.

■ Pro Tip

*Use the phrase **'layer by layer'** — it signals a calm, systematic engineer.*

QUESTION #48

■ Q48. Tell me about a time you handled a production crash. What was your process?

■ Why Interviewers Ask This

Behavioral question that tests calm under pressure.

■ Strong Candidate Answer

Yes — once we shipped a release that crashed on Android 9 devices on startup. **Sentry** alerted me within minutes with the stack trace, and I saw it was a null pointer in a native library that didn't support that OS. First thing: I **rolled back** the Play Store release to the previous version, so new users got the working build. Second: I reproduced the crash locally on an Android 9 emulator. Third: I added a **version check** guard and shipped a hotfix within a few hours. Final step: I added a regression test and a Sentry alert for that specific error so we'd catch it earlier next time.

■ Pro Tip

*The four steps — **rollback** → **reproduce** → **fix** → **prevent** — make any incident answer sound senior.*

QUESTION #49

■ Q49. How do you communicate a technical decision to a non-technical stakeholder?

■ Why Interviewers Ask This

Soft-skill question — critical even at junior levels.

■ Strong Candidate Answer

I lead with the **impact on them**, not the technology. So instead of saying 'I want to migrate to TypeScript,' I'd say 'we're hitting recurring production bugs that hurt our users, and there's a one-time investment we can make that will reduce those by maybe 70% over the next quarter.' Then if they want detail, I go deeper. I also use **analogies** — for caching I'll say 'it's like keeping a snack on your desk so you don't walk to the kitchen every time.' And I always quantify trade-offs: this costs us two weeks now, saves us a week every month after.

■ Pro Tip

*Always lead with **user impact and numbers**, not technology — that's how engineers earn trust with PMs and execs.*

QUESTION #50

■ Q50. How do you stay up to date with new AI tools and React Native changes?

■ Why Interviewers Ask This

Behavioral — shows curiosity and learning habits.

■ Strong Candidate Answer

Honestly, I treat learning as part of the job. I follow the **React Native release notes** and the **Expo blog**, because both ecosystems move fast. For AI, I read the **Anthropic and OpenAI changelogs** directly — that's where features like streaming and tool calling are announced. I also use the tools — **Claude Code**, **Cursor**, **Copilot** — every day, and I've noticed they each have strengths. Beyond that, I rebuild a small side project every quarter using whatever's new, because I learn fastest by building, not just reading. I also share what I learn with my team in short demos.

■ Pro Tip

*Mention you **share with the team** — interviewers love candidates who teach, not just hoard knowledge.*

9 Section 9 — AI Streaming, Chat & Cost Optimization

Server-Sent Events, cancel handling, prompt caching, context window management, partial JSON, cost estimation, latency budgets.

QUESTION #51

■ Q51. How does Server-Sent Events (SSE) actually work, and why is it preferred for LLM streaming?

■ Why Interviewers Ask This

Tests your understanding of the protocol behind modern AI streaming APIs.

■ Strong Candidate Answer

SSE is a one-way streaming protocol that runs over plain **HTTP**. The client opens a normal GET or POST, and the server keeps the connection open and pushes messages as text — each one formatted as **data: {json}** followed by a blank line. The browser or native fetch parses it line by line. It's preferred for LLM streaming because it works through every **HTTP proxy and load balancer** in the wild, supports **auto-reconnect** with EventSource, and is much simpler than WebSockets for a server-push use case. Both **Anthropic** and **OpenAI** use it for their streaming chat endpoints.

■ Pro Tip

*Mention that SSE is **one-way only** — that's why we fire a fresh request per message instead of using a long-lived bidirectional channel.*

QUESTION #52

■ Q52. How do you handle a streaming AI connection that drops mid-response?

■ Why Interviewers Ask This

Real-world reliability question — mobile networks fail constantly.

■ Strong Candidate Answer

First, detect the failure — the stream's **error** or **close** event fires, or no chunks arrive for a timeout window like 10 seconds. Then I keep the **tokens we already received** and surface them in the bubble with a small **retry** button. If the user taps retry, I resend the conversation history with the **partial assistant message** appended and a prompt to continue from where it left off. I never silently drop the message. On flaky cellular, I also do a single auto-retry with backoff before showing the manual retry.

■ Pro Tip

*Always say **'never silently drop'** — losing a chat message is one of the worst UX bugs in AI apps.*

QUESTION #53

■ Q53. How would you implement a 'Stop' button while a Claude or OpenAI response is streaming?

■ Why Interviewers Ask This

Tests fetch and async cancellation knowledge.

■ Strong Candidate Answer

I create an **AbortController** when I start the request and pass its **signal** into fetch. When the user taps stop, I call **controller.abort()** — the fetch rejects, the stream closes, and the server stops generating shortly after. I keep whatever tokens already arrived in the chat bubble and swap the 'Stop' button for a 'Regenerate' button. One honest detail: you still get billed for tokens that arrived before the abort, but both Anthropic and OpenAI **stop generation server-side** on disconnect, so you don't pay for the full unwritten response.

■ Pro Tip

*Mention the **billing implication** — interviewers notice when candidates think about cost, not just code.*

QUESTION #54

■ Q54. What is prompt caching and how does it save you money on the Claude API?

■ Why Interviewers Ask This

Tests knowledge of a major cost-optimization feature.

■ Strong Candidate Answer

Prompt caching lets you mark parts of a prompt — usually the **system prompt**, **RAG documents**, or **tool definitions** — to be cached server-side. On the next request with the same cached prefix, Anthropic charges roughly **10% of normal input cost** for those tokens and serves them with **much lower first-token latency**. You enable it by adding **cache_control: { type: 'ephemeral' }** on the relevant content blocks. It's a huge win for chat apps with long static system prompts — I've seen **80%+ cost reduction** in production with no UX change.

■ Pro Tip

*If asked when not to use it, mention that caching has a **small write overhead** — it only pays off if the same prefix is reused within minutes.*

QUESTION #55

■ Q55. How do you manage the context window when a chat conversation gets very long?

■ Why Interviewers Ask This

Practical AI engineering question many juniors miss.

■ Strong Candidate Answer

Every model has a token limit — Claude is around 200k — and you can't just keep appending forever, both for the limit and the cost per turn. I track **token count** as I go using the Anthropic SDK or tiktoken. When I'm around 70% of the window, I start **summarizing older turns**: I ask the model to compress everything before message N into a short summary, then replace those messages with the summary. The user keeps the feel of long memory while the actual context stays small. For simpler apps I use a **sliding window** — system prompt plus the last 20 turns.

■ Pro Tip

*Use the phrase **'rolling summary'** or **'sliding window'** — naming the pattern signals seniority.*

QUESTION #56

■ Q56. How do you parse partial JSON when an LLM streams a tool call back to you?

■ Why Interviewers Ask This

Tests deep streaming knowledge.

■ Strong Candidate Answer

Tool calls stream **character by character**, so for most of the stream you have invalid JSON like **{"city": "new yor**. Two strategies. The simple one: wait until the streaming event signals **tool_use complete** (Anthropic fires a **content_block_stop**), then parse the assembled string with normal JSON.parse. The live one: use a library like **partial-json** or **jsonrepair** that handles incomplete JSON — useful when you want to show 'AI is calling getWeather for…' in real time. I default to the complete-then-parse approach unless I need progressive UI.

■ Pro Tip

Mention both approaches — 'wait and parse' vs 'partial parse' — interviewers like trade-off reasoning.

QUESTION #57

■ Q57. What's the difference between system, user, and assistant messages in a chat API?

■ Why Interviewers Ask This

Beginner-friendly fundamentals.

■ Strong Candidate Answer

Every modern chat API uses three roles. The **system message** sets the model's behavior — 'you are a helpful coding assistant' — and the model is trained to weight it strongly. The **user message** is what the human typed. The **assistant message** is what the model previously said, included in history so it remembers the conversation. Order matters: typically system first, then alternating user and assistant. One important security rule: **never put untrusted user input into the system message** — that's how **prompt injection** attacks work. The user's text always goes in the user role.

■ Pro Tip

*The **prompt injection** point at the end always lands — interviewers love when you connect features to security.*

QUESTION #58

■ Q58. How do you render markdown smoothly while text is still streaming in?

■ Why Interviewers Ask This

Tests practical UI performance under streaming.

■ Strong Candidate Answer

Two challenges. First, **re-render performance**: if you re-parse the full markdown on every token, you redraw 50 times per second. I throttle the render to about **60ms** — visually identical but much cheaper. Second, **incomplete syntax**: mid-stream you'll have unclosed bold like ****hello wor**, which renders ugly. Some libraries handle this gracefully; if not, I run a tiny **auto-close pass** that closes any open formatting before render. Libraries: **react-native-markdown-display** on RN, **react-markdown** on web.

■ Pro Tip

*Mention **throttling re-renders** — it's the small performance detail that proves you've actually built a streaming UI.*

QUESTION #59

■ Q59. How would you estimate the cost of an AI chat feature before shipping it?

■ Why Interviewers Ask This

Tests product and engineering judgment.

■ Strong Candidate Answer

I build a small calculator. For each **session**, estimate average **input tokens** (system prompt + history) and average **output tokens** (the reply). Multiply each by the per-million-token price for the chosen model. Then estimate **sessions per user per day** and **active users**. So: cost per session × sessions per user × users = daily cost. I always factor in a **cache hit ratio** if using prompt caching, plus a buffer for **retries and tool round-trips**. Before launch, I run a small beta to validate the assumptions — real-world token counts are usually higher than estimates.

■ Pro Tip

*Always end with **'validate with a beta'** — it shows you trust real data over spreadsheets.*

QUESTION #60

■ Q60. What’s a typical latency budget for a 'feels real-time' AI chat on mobile?

■ Why Interviewers Ask This

Tests UX intuition around perceived performance.

■ Strong Candidate Answer

The number that matters most is **TTFT — time to first token**. Under **500ms** feels instant; under **1 second** is acceptable; over **2 seconds** feels broken. After the first token, as long as tokens flow at **20+ per second**, users feel it’s smooth. To hit that budget I keep the system prompt short, enable **prompt caching**, pick a model in a **nearby region**, and stream from token one. Spinners that last more than 800ms are where users start tapping the back button.

■ Pro Tip

*The phrase **'spinners over 800ms lose users'** is memorable — interviewers often quote it back.*

10 Section 10 — Whisper, Voice AI & Audio Pipeline

Audio formats, sample rates, VAD libraries, AVAudioSession, echo cancellation, TTS choice, end-to-end voice latency.

QUESTION #61

■ Q61. What audio format should you send to Whisper, and why does it matter?

■ Why Interviewers Ask This

Beginner-friendly question on a detail that affects latency and cost.

■ Strong Candidate Answer

Whisper accepts **mp3, m4a, wav, webm, mp4, mpeg, mpga**. The trick is choosing one based on your constraint. On cellular, I use **m4a (AAC)** because the upload is small — a 10-second clip might be 30KB instead of 300KB for WAV. On Wi-Fi or when latency matters most, I use **16kHz mono WAV** because Whisper internally resamples to that anyway, so no server-side transcoding step. Bad choice: high-sample-rate stereo WAV — it's 10x bigger than needed and Whisper just downsamples it.

■ Pro Tip

*Always mention **'Whisper resamples to 16kHz mono internally'** — it's the one detail that justifies your format choice.*

QUESTION #62

■ Q62. What sample rate and channel count does Whisper actually want?

■ Why Interviewers Ask This

Tests detail-level voice AI knowledge.

■ Strong Candidate Answer

Whisper internally works at **16kHz mono**. So if you can encode in 16kHz mono on the device, you skip a transcoding step server-side and trim a few hundred milliseconds. Higher sample rates like 44.1kHz or 48kHz still work — Whisper just downsamples — but you waste bandwidth and a tiny bit of latency. Stereo gets mixed to mono. Voice doesn't benefit from stereo for transcription, so I always record mono.

■ Pro Tip

*Mention **'mono saves 50% bandwidth'** — it's the kind of micro-optimization interviewers love.*

QUESTION #63

■ Q63. How do you keep Whisper transcription latency low for short utterances?

■ Why Interviewers Ask This

Tests practical voice AI tuning.

■ Strong Candidate Answer

Four levers. First, **compress the audio** — m4a uploads way faster than WAV. Second, **trim silence with VAD** before sending, so Whisper isn't transcribing empty space. Third, **chunk long audio** — send 5-second windows and stitch transcripts, instead of one big request at the end. Fourth, **region selection** — use a Whisper endpoint geographically close to the user. With those, I usually hit **300–600ms** for short phrases. If you need sub-200ms, look at **Deepgram** or **AssemblyAI realtime** over WebSocket — those give you partial transcripts live.

■ Pro Tip

*Name a **specific alternative** like Deepgram — it shows you've evaluated trade-offs, not just used the default.*

QUESTION #64

■ Q64. What's the difference between Silero VAD and WebRTC VAD?

■ Why Interviewers Ask This

Beginner-friendly question on tools you mentioned.

■ Strong Candidate Answer

WebRTC VAD is the classic — small, written in C, runs everywhere, very fast. But it's based on signal-processing heuristics, so it produces **false positives in noisy environments** — wind, traffic, fan hum can all trigger 'speech detected.' **Silero VAD** is a small neural network — slightly heavier, but dramatically more accurate, especially in real-world noise. For mobile voice AI I default to Silero because false positives mean sending silence to Whisper, which costs money and adds latency. Both run on-device, so no privacy concerns.

■ Pro Tip

*Always mention **'runs on-device'** when comparing VAD — privacy and cost both matter for voice features.*

QUESTION #65

■ Q65. What is AVAudioSession on iOS and why does it matter for voice apps?

■ Why Interviewers Ask This

Tests iOS-specific audio knowledge.

■ Strong Candidate Answer

AVAudioSession is iOS's system-wide audio configuration. It defines how your app's audio interacts with the rest of the device — does it mix with music, does it activate the speaker, can it record while playing, does it work over Bluetooth. The big decision is the **category**: **playAndRecord** for voice apps, **playback** for video. Then the **mode**: **voiceChat** for VOIP-style apps unlocks **built-in echo cancellation** and routes through the earpiece by default. Wrong category and your app records but can't play, or vice versa.

■ Pro Tip

*Mention **'voiceChat unlocks echo cancellation'** — that's the specific detail that proves you've shipped a voice app.*

QUESTION #66

■ Q66. How do you handle echo cancellation when TTS and mic are both active?

■ Why Interviewers Ask This

Tests deep voice AI knowledge.

■ Strong Candidate Answer

Without echo cancellation, the mic picks up the TTS audio coming out of the speaker and re-sends it to Whisper, creating a **feedback loop** where the AI hears its own voice. The fix on **iOS** is using AVAudioSession in **mode voiceChat** or **videoChat** — both turn on the built-in AEC. On **Android**, wrap your AudioRecord with the **AcousticEchoCanceler** API if the device supports it. WebRTC libraries also bundle their own AEC if you need cross-platform. With AEC on, the user can talk over the AI naturally — no feedback.

■ Pro Tip

*End with **'user can talk over the AI'** — that's the user-visible benefit and the reason it matters.*

QUESTION #67

■ Q67. How do you pick the right TTS voice — what trade-offs matter?

■ Why Interviewers Ask This

Tests product judgment around AI features.

■ Strong Candidate Answer

Three axes: **quality, latency, cost**. **ElevenLabs** is the gold standard — voices sound human, supports cloning — but slowest and most expensive. **OpenAI TTS** is excellent quality, faster, much cheaper, good enough for most chat assistants. **System TTS** — iOS AVSpeechSynthesizer, Android TextToSpeech — is **free, instant, runs offline**, but sounds robotic. For an AI assistant where character matters I'd use ElevenLabs or OpenAI; for utility apps reading short messages, system TTS is fine. Always **stream audio in chunks** so playback starts before the full file is generated.

■ Pro Tip

*Mention **'system TTS is offline'** — it's the killer feature interviewers forget exists.*

QUESTION #68

■ Q68. What's a typical end-to-end latency budget for a voice AI turn?

■ Why Interviewers Ask This

Tests holistic understanding of a voice pipeline.

■ Strong Candidate Answer

Breaking it down: **mic stop to VAD-detected end** is 100–300ms; **audio upload + Whisper** is 300–800ms; **LLM time-to-first-token** is 300–700ms; **TTS first audio chunk** is 200–500ms. Sum it up and a great voice AI app feels real-time at **1–2 seconds end-to-end**. Under 1 second is exceptional. Over 3 seconds, users disengage. Every component matters — shaving 200ms off Whisper by switching to **streaming transcription** often gives the biggest perceived improvement.

■ Pro Tip

*Mention **'streaming transcription saves the most'** — it's the lever with the biggest user-visible win.*

---

11 Section 11 — Tool Calling, Agents & Safety

Tool schemas, validation, multi-step agent loops, dangerous-tool guardrails, prompting techniques.

QUESTION #69

■ Q69. How do you define a tool schema for Claude or OpenAI tool calling?

■ Why Interviewers Ask This

Tests practical agent implementation.

■ Strong Candidate Answer

A tool definition is a small **JSON Schema** with three things: a **name**, a clear **description**, and a **parameters** object describing inputs. Example: name **getWeather**, description **'Get current weather for a city. Returns temperature in Celsius and condition.'**, parameters **{ city: { type: 'string', description: 'City name in English' } }**. The descriptions matter more than people realize — the model uses them to **decide which tool to call**, so vague descriptions lead to wrong tool selection. Keep the tool count small — under 20 — because too many confuses the model.

■ Pro Tip

*The phrase **'descriptions matter more than people realize'** is something interviewers nod to — most candidates skip it.*

QUESTION #70

■ Q70. What happens if the LLM picks the wrong tool or calls it with invalid arguments?

■ Why Interviewers Ask This

Tests error-handling instincts for agents.

■ Strong Candidate Answer

Three failure modes I plan for. **Invalid arguments** — I validate every tool input with **zod** server-side; if it fails, I return the validation error as the tool result and the model usually retries with corrected args. **Wrong tool entirely** — I just run it and return the result; if it's nonsensical, the model self-corrects on the next turn. **Tool execution error** — I return the actual error message in the tool_result, not a generic 'failed.' I also **log every tool call** with inputs and outputs so I can see which prompts cause misbehavior and improve the descriptions.

■ Pro Tip

*Mention **'log every tool call'** — observability for agents is what separates production from demo.*

QUESTION #71

■ Q71. How do you build a multi-step agent loop without it running forever?

■ Why Interviewers Ask This

Tests safety and resource awareness.

■ Strong Candidate Answer

An agent loop is: model picks tool → execute → return result → model picks next → repeat. Two safety guards are essential. **Max iterations** — I cap at 10 or 15 turns; if not done by then, something's wrong. **Max wall time** — say 30 or 60 seconds total; anything longer is probably a loop. I also **stream intermediate steps to the UI** so the user sees 'searching files…', 'reading X…', 'thinking…' — they can hit cancel any time. And I log the full transcript so I can debug runaway loops after the fact.

■ Pro Tip

*Mention **'stream steps to the UI'** — agent UX is mostly about making the wait feel intentional.*

QUESTION #72

■ Q72. How would you handle a dangerous tool — like 'delete account' — safely in an AI agent?

■ Why Interviewers Ask This

Tests safety design.

■ Strong Candidate Answer

Never let an AI execute destructive actions silently. The pattern I use: classify tools as **read**, **write**, or **destructive**. Read tools auto-run. Write tools may auto-run or require confirmation depending on impact. **Destructive tools always require explicit user confirmation** — when the model calls them, I pause the loop, show a confirmation UI with the exact action and arguments, and only proceed on user tap. I also support a **dry-run mode** where the tool returns 'this is what would happen' without doing it. And I always **audit log** destructive tool calls with user, time, args.

■ Pro Tip

*Use the phrase **'explicit confirmation for destructive'** — it's the universal AI safety mantra.*

QUESTION #73

■ Q73. What’s the difference between zero-shot, few-shot, and ReAct prompting?

■ Why Interviewers Ask This

Tests prompting vocabulary.

■ Strong Candidate Answer

Zero-shot is just asking the model directly with no examples — 'classify this review as positive or negative.' Works well for modern models on common tasks. **Few-shot** means including **2–5 example input/output pairs** in the prompt to teach the format — useful when output needs strict structure. **ReAct** stands for Reasoning + Acting — the model alternates between **thinking out loud** and **taking actions** (tool calls), which improves accuracy on multi-step tasks. Modern Claude and GPT do most things well zero-shot; I reach for few-shot mainly for strict formatting and ReAct for complex agents.

■ Pro Tip

*Mention you **default to zero-shot and add complexity only when needed** — it shows pragmatism over cargo-culting.*

12 Section 12 — Native Modules: Bridge, JSI & TurboModules

Old bridge vs new architecture, JSI, TurboModules, Fabric, Kotlin/Swift bridging, ViewManagers, autolinking, debugging native crashes, memory & threads.

QUESTION #74

■ Q74. What is the React Native bridge, and why is it being replaced?

■ Why Interviewers Ask This

Tests fundamental architecture knowledge.

■ Strong Candidate Answer

The old **bridge** is the communication layer between the **JS thread** and the **native UI thread**. Every call between them goes through a **JSON-serialized, asynchronous, batched queue**. It worked but had real downsides: serialization overhead for high-frequency calls (animations, gestures, lots of native module calls), and everything was asynchronous — you couldn't call a native function and get a value back synchronously. The new architecture replaces this with **JSI (JavaScript Interface)**, which lets JS and native talk directly in memory.

■ Pro Tip

*Always frame it as **'old async-batched bridge vs new direct JSI'** — that contrast is the one-liner interviewers want.*

QUESTION #75

■ Q75. What is JSI and how is it different from the old bridge?

■ Why Interviewers Ask This

Tests new architecture knowledge.

■ Strong Candidate Answer

JSI stands for **JavaScript Interface**. It's a C++ layer that lets the JS engine — **Hermes** or JavaScriptCore — call host (native) functions directly, and vice versa, **without JSON serialization**. So instead of every native module call going through a queue and being serialized, you can register a C++ function as a JS property and call it like a normal JS function. This unlocks **synchronous calls**, much lower overhead, and is the foundation for **TurboModules**, **Fabric**, and modern **Reanimated**. You don't usually write raw JSI code — you use TurboModules or libraries that wrap it.

■ Pro Tip

*Mention **Hermes** by name — it's the modern JS engine and the standard pairing with JSI.*

QUESTION #76

■ Q76. What are TurboModules and what problems do they solve?

■ Why Interviewers Ask This

Tests new architecture knowledge in depth.

■ Strong Candidate Answer

TurboModules are the new architecture's replacement for native modules, built on top of JSI. They solve three problems. **Lazy loading** — modules initialize only when first used, so app startup is faster. **Type safety via codegen** — you describe your module's interface in a TypeScript-ish spec, and codegen generates both the JS bindings and the native interface, so types are guaranteed to match. **Synchronous methods** — you can call a native method and get the value back immediately, no Promise needed. Migrating an existing native module to TurboModule is mostly mechanical.

■ Pro Tip

*Mention **'codegen guarantees JS-native types match'** — this is the killer feature that prevents a whole class of bugs.*

QUESTION #77

■ Q77. What is Fabric in the React Native new architecture?

■ Why Interviewers Ask This

Tests rendering architecture knowledge.

■ Strong Candidate Answer

Fabric is the new rendering system. It replaces the old **Shadow Tree** with a JSI-based one that lives in C++, shared between threads. Two big wins. **Synchronous layout** — components can measure themselves immediately without a bridge round-trip, which fixes a class of jank. **Concurrent React features** — Suspense, transitions, and the priority scheduler work properly because React can schedule work without fighting the bridge. Combined with TurboModules and JSI, this is what people mean by 'the new architecture' that rolled out around 2023.

■ Pro Tip

*Mention **'concurrent React features work properly'** — it ties Fabric to features web devs already know.*

QUESTION #78

■ Q78. How do you expose a Kotlin function to JavaScript as a Promise?

■ Why Interviewers Ask This

Tests practical native module implementation.

■ Strong Candidate Answer

In a class extending **ReactContextBaseJavaModule**, I write a function annotated with **@ReactMethod** that takes a **Promise** parameter at the end. Inside, I do the work and call **promise.resolve(value)** on success or **promise.reject(code, message)** on failure. If the work is async, I launch a **coroutine** with a CoroutineScope and resolve from inside. Example: **@ReactMethod fun fetchData(promise: Promise) { scope.launch { try { val r = api.fetch(); promise.resolve(r) } catch (e: Exception) { promise.reject('FETCH_ERR', e) } }**. On the JS side this becomes a regular async function returning a Promise.

■ Pro Tip

Be ready to write this on a whiteboard — interviewers often ask for the actual code, not just the concept.

QUESTION #79

■ Q79. How do you bridge Swift async/await to a JavaScript Promise?

■ Why Interviewers Ask This

Tests iOS native module knowledge.

■ Strong Candidate Answer

An iOS native module method takes **RCTPromiseResolveBlock** and **RCTPromiseRejectBlock** as its last two arguments. Inside, I wrap the async call in a **Task { ... }** block and call resolve or reject when done. Example: **@objc func fetchData(_ resolve: @escaping RCTPromiseResolveBlock, rejecter reject: @escaping RCTPromiseRejectBlock) { Task { do { let r = try await api.fetch(); resolve(r) } catch { reject('FETCH_ERR', error.localizedDescription, error) } } }**. In the new architecture with TurboModules, you can declare async methods more directly via codegen.

■ Pro Tip

*Mention **'always call resolve OR reject, never both, never neither'** — that's the gotcha that causes hung Promises.*

QUESTION #80

■ Q80. How do you emit events from native code back to JavaScript?

■ Why Interviewers Ask This

Tests bidirectional communication.

■ Strong Candidate Answer

I use **DeviceEventEmitter** or **NativeEventEmitter**. On the native side I call something like **reactContext.getJSModule(RCTDeviceEventEmitter::class.java).emit('audioChunk', data)** on Android, or **sendEvent(withName: 'audioChunk', body: data)** on iOS. On the JS side, I subscribe with **new NativeEventEmitter(MyModule).addListener('audioChunk', handler)**. This is how I stream **audio frames, location updates, BLE scans** — anything that produces a stream of values rather than a single result. Important: always **remove the listener** in cleanup or you leak memory.

■ Pro Tip

*Always mention **'remove the listener on cleanup'** — forgetting it is the #1 native module memory leak.*

QUESTION #81

■ Q81. What is a ViewManager and when would you need one?

■ Why Interviewers Ask This

Tests native UI component knowledge.

■ Strong Candidate Answer

A **ViewManager** exposes a native UI component to React Native — not just a function but an actual on-screen view. You subclass **SimpleViewManager** on Android or **RCTViewManager** on iOS, return the native view, and map **JSX props** to native properties using annotations like **@ReactProp**. You need one when wrapping a third-party native widget — a custom **map view**, a **video player**, a **camera preview**, or any vendor SDK that ships a UIView. Examples: **react-native-maps** and **react-native-vision-camera** are both ViewManagers under the hood.

■ Pro Tip

*Name **react-native-maps** or **vision-camera** — interviewers instantly recognize them.*

QUESTION #82

■ Q82. What is autolinking in React Native and what does it do for you?

■ Why Interviewers Ask This

Beginner-friendly question on tooling.

■ Strong Candidate Answer

Autolinking is the system that automatically wires up native dependencies after you **npm install** a React Native library. Before autolinking — pre 0.60 — you had to manually edit **Podfile** on iOS and **settings.gradle** on Android, run **react-native link**, and pray. Now, when you install a library, React Native scans **node_modules** for **react-native.config.js** or known patterns and adds the native code to the build automatically. You just do **cd ios && pod install** on iOS and rebuild. It saves hours of setup and is the reason RN went from painful to pleasant for adding libraries.

■ Pro Tip

*Mention **'pod install still needed on iOS'** — it's the one manual step people forget.*

QUESTION #83

■ Q83. How would you debug a native module that's crashing on Android?

■ Why Interviewers Ask This

Tests practical debugging skills.

■ Strong Candidate Answer

Step one: **adb logcat | grep -E 'AndroidRuntime|FATAL'** — the crash trace shows up there immediately with exception and stack. If it's a Kotlin/Java exception, the line number points right at the bug. For native (C++) crashes with no symbols, I **symbolicate** using the build's mapping file. For deeper debugging I attach the **Android Studio debugger** to the running app, set breakpoints in the native module, and step through. In production, **Sentry** or **Crashlytics** capture the same trace with source maps. The mistake juniors make is reading only the JS error — most native crashes don't show up in the JS console.

■ Pro Tip

*Mention **'most native crashes don't show in the JS console'** — it's the most common reason juniors get stuck.*

QUESTION #84

■ Q84. How do you avoid memory leaks in a native module?

■ Why Interviewers Ask This

Tests real-world native development care.

■ Strong Candidate Answer

Three usual suspects. **Event listeners** — every NativeEventEmitter listener on the JS side and every native callback registration must be paired with a cleanup, otherwise they hold references forever. **Holding ReactContext or Activity references** in long-lived objects — they become invalid when the activity recreates and prevent garbage collection. **Bitmaps and large native buffers** on Android need explicit recycling or try-with-resources. I profile with **Android Studio Profiler** or **Xcode Instruments** — they highlight leaks visually. A leak of 200KB per session adds up fast across a million users.

■ Pro Tip

*Mention **'200KB per session adds up fast'** — concrete numbers make abstract leak talk land.*

QUESTION #85

■ Q85. How do you keep a native module thread-safe?

■ Why Interviewers Ask This

Tests concurrency awareness.

■ Strong Candidate Answer

First, know which thread you're on. **@ReactMethod** on Android runs on a **native modules thread pool** — not the UI thread. So if I need to touch the UI, I wrap it in **UiThreadUtil.runOnUiThread { ... }**. On iOS, similar — use **DispatchQueue.main.async**. For shared mutable state, I use **synchronized** blocks or a **Mutex** in Kotlin, and **DispatchQueue** or **NSLock** on Swift. And the Promise rule: **resolve or reject exactly once**, from any thread. Calling it twice or never causes hung Promises that are a nightmare to debug.

■ Pro Tip

*Mention **'resolve exactly once'** — it's the specific concurrency bug that hits everyone eventually.*

13 Section 13 — Mobile Platform Integration

FCM setup, APNs, deep links, runtime permissions, in-app purchases, app lifecycle, secure storage, safe area, splash screens.

QUESTION #86

■ Q86. How do you set up FCM end-to-end for a React Native app?

■ Why Interviewers Ask This

Tests practical push setup knowledge.

■ Strong Candidate Answer

Roughly six steps. **Create a Firebase project**. Add the Android app, download **google-services.json**, drop into **android/app/**. Add the iOS app, download **GoogleService-Info.plist**, drag into Xcode, and upload your **APNs auth key (p8)** to Firebase. Install **@react-native-firebase/app** and **@react-native-firebase/messaging**. In code: request notification permission, get the **FCM token** with messaging().getToken(), send it to my backend. Set up handlers for **foreground**, **background**, and **killed** states — they're different APIs. Test with a curl to Firebase's send endpoint before integrating fully.

■ Pro Tip

*Mention testing with a **curl call to Firebase** before backend work — it isolates whether the issue is mobile or server.*

QUESTION #87

■ Q87. What's the difference between APNs certificates and auth keys, and which should you use?

■ Why Interviewers Ask This

Tests iOS push knowledge.

■ Strong Candidate Answer

Both let your server send push notifications to iOS devices. **Certificates** (the older approach) expire **every year**, are tied to a single app, and are different for dev and prod — so you rotate them annually and manage multiple files. **Auth keys (p8 files) never expire**, work for **all apps under your team**, and handle both dev and prod. **Always use auth keys** in modern projects — less maintenance, fewer outages from expired certs. I've seen production push fail at 2am because someone forgot to renew a cert.

■ Pro Tip

*The **'2am production outage'** story is the kind of war-story detail interviewers remember.*

QUESTION #88

■ Q88. What's the difference between URL schemes, Universal Links, and App Links?

■ Why Interviewers Ask This

Tests deep linking knowledge.

■ Strong Candidate Answer

All three let an external trigger open your app to a specific screen. **URL schemes** like **myapp://order/123** are simple but **unverified** — any app can register the same scheme and hijack it. **Universal Links (iOS)** are real **https URLs** verified via an **apple-app-site-association** file hosted on your domain. **App Links (Android)** are the equivalent with **assetlinks.json**. The big win of Universal/App Links: they open your app if installed, otherwise fall back to your website seamlessly. I always go with Universal/App Links in production.

■ Pro Tip

*Mention **'fall back to website if not installed'** — it's the killer UX feature URL schemes can't match.*

QUESTION #89

■ Q89. How do you request and check runtime permissions on iOS and Android?

■ Why Interviewers Ask This

Beginner-friendly platform basics.

■ Strong Candidate Answer

On **iOS**, you declare every permission in **Info.plist** with a **usage string** explaining why — without that string, the app crashes when requesting. The OS shows the popup the first time you ask. On **Android**, declare in **AndroidManifest.xml**; for 'dangerous' permissions (camera, location, mic, contacts), you also request at runtime. I use **react-native-permissions** for a unified API. Always **check before requesting** — if it's already granted, skip the popup. And handle the **'denied permanently'** case by linking to system settings.

■ Pro Tip

*Mention **'linking to settings on permanent denial'** — it's the UX path most apps miss.*

QUESTION #90

■ Q90. How do In-App Purchases work and what's the difference between StoreKit and Play Billing?

■ Why Interviewers Ask This

Tests monetization fundamentals.

■ Strong Candidate Answer

Apps selling digital goods **must** use Apple's **StoreKit** on iOS and Google's **Play Billing** on Android — both take a 15–30% cut. Each supports **consumables** (coins), **non-consumables** (unlock a feature), and **subscriptions**. I use **react-native-iap** to bridge both behind one API. Critical: always **validate the receipt server-side** against Apple/Google's servers — client-only validation can be faked, leading to free unlocked content. You **cannot use Stripe or PayPal** for digital goods — only for physical goods, real-world services, or B2B.

■ Pro Tip

*Always mention **'server-side receipt validation'** — this is the IAP-specific security must.*

QUESTION #91

■ Q91. How do you handle the app foreground / background transitions?

■ Why Interviewers Ask This

Tests lifecycle awareness.

■ Strong Candidate Answer

React Native exposes **AppState** with three states: **active**, **background**, **inactive** (a brief transitional state on iOS). I subscribe with **AppState.addEventListener('change', handler)**. On **background**, I **save state to disk**, close streaming connections to save battery, pause timers. On **active**, I reconnect, refresh stale data, check for missed notifications. On iOS the app **can be killed any time** in background, so I never assume state persists in memory. I also use this hook to track **session length** for analytics.

■ Pro Tip

*Mention **'iOS can kill the app any time'** — it's the assumption juniors get wrong most often.*

QUESTION #92

■ Q92. How do you store a JWT or refresh token securely on a mobile device?

■ Why Interviewers Ask This

Tests security awareness.

■ Strong Candidate Answer

Never put it in **AsyncStorage** — that's plain text on Android and easy to read on a rooted device. Use the platform's secure store: **Keychain on iOS** and **EncryptedSharedPreferences** or the **Keystore** on Android. The library I use is **react-native-keychain**, which abstracts both. For extra paranoia I store the **refresh token** in the secure store and keep the short-lived **access token** only in memory — that way even a process dump doesn't expose long-term credentials. Also, set the keychain accessibility to **after-first-unlock** so it's not accessible until the user unlocks the device.

■ Pro Tip

*Mention **'after-first-unlock'** — that's the keychain detail that signals real security awareness.*

QUESTION #93

■ Q93. What is SafeAreaView and why do you need it?

■ Why Interviewers Ask This

Beginner-friendly UI question.

■ Strong Candidate Answer

Modern phones have **non-rectangular display areas** — the iPhone notch, the home indicator at the bottom, status bars, Android cutouts. If you render content edge-to-edge without thinking, your buttons sit under the notch and your text gets clipped. **SafeAreaView** wraps content so it stays inside the visible, untouchable area. The current best library is **react-native-safe-area-context**, which provides hooks like **useSafeAreaInsets()** giving you exact pixel insets for each side. The old built-in SafeAreaView is iOS-only and deprecated.

■ Pro Tip

*Mention **'old SafeAreaView is iOS-only'** — many candidates don't realize they should use the modern library.*

QUESTION #94

■ Q94. How do you handle the iPhone notch, Dynamic Island, and Android cutouts?

■ Why Interviewers Ask This

Tests detail-level UI awareness.

■ Strong Candidate Answer

Same toolkit: **react-native-safe-area-context** works on iOS and Android — it gives the inset values for each edge. The **Dynamic Island** sits inside the iOS top safe area, so SafeAreaView protects you automatically — but I avoid putting critical UI in the top 60pt anyway because system overlays (recording, calls) can cover it. For Android cutouts, declare **layoutInDisplayCutoutMode** in styles.xml and let SafeAreaView handle the rest. Always **test on real devices** with and without notches — emulators sometimes lie.

■ Pro Tip

*Mention **'always test on real notch devices'** — many candidates only check the simulator and miss visual bugs.*

QUESTION #95

■ Q95. How do you set up a custom splash screen and app icon for both platforms?

■ Why Interviewers Ask This

Beginner-friendly question on shipping basics.

■ Strong Candidate Answer

For **icons**, I use a generator like **@bam.tech/react-native-make** or Expo's image-utils — feed one 1024×1024 PNG, it spits out every size each platform needs. On **iOS**, the icons go in the Asset Catalog; on **Android**, into **res/mipmap-*** folders. For **splash screens** I use **react-native-bootsplash** (or expo-splash-screen if Expo). It generates the native assets, configures iOS **LaunchScreen.storyboard** and Android **splash theme + drawable**, and hides the splash from JS when the app is ready. Critical: don't show a long fake splash to hide a slow startup — fix the startup instead.

■ Pro Tip

*Mention **'fix slow startup, don't hide it'** — it's the perspective that separates engineers from icon-pushers.*

14 Section 14 — Real-time AI on Mobile & On-Device AI

WebSocket reconnect, background-stream handling, battery, Core ML / TFLite / ONNX, on-device vs cloud trade-offs.

QUESTION #96

■ Q96. How do you reconnect a WebSocket streaming AI session after a network drop?

■ Why Interviewers Ask This

Tests resilience design.

■ Strong Candidate Answer

Three pieces. First, **detect the drop** — both onClose and onError fire, plus a heartbeat ping to detect dead connections that haven't closed. Second, **reconnect with exponential backoff** — wait 1s, 2s, 4s, up to a cap of 30s, with a bit of **jitter** to avoid thundering herd. Third, **resume meaningfully** — the server holds a session ID, and on reconnect the client sends 'resume from message N' so the AI continues instead of restarting. During reconnect, show a **discreet 'reconnecting…' banner**, not a blocking modal — users hate modals.

■ Pro Tip

*Mention **'jitter to avoid thundering herd'** — it's the small detail that shows distributed-systems awareness.*

QUESTION #97

■ Q97. What happens to a streaming AI request when the user backgrounds the app?

■ Why Interviewers Ask This

Tests deep lifecycle knowledge.

■ Strong Candidate Answer

On **iOS**, the app typically gets about **30 seconds** before being suspended; long network requests usually fail mid-stream. On **Android** it varies but is similar without a foreground service. Strategy: when the app backgrounds, I **complete the request on the server** (so the AI finishes its reply) and store the result. When the app foregrounds, I **fetch the completed message** and render it. Alternative: send a **silent push notification** when the response is ready, so the user knows to come back. Never assume a streaming connection survives backgrounding.

■ Pro Tip

*End with **'never assume the stream survives backgrounding'** — it's the rule that prevents nasty production bugs.*

QUESTION #98

■ Q98. What are the battery implications of long-running AI sessions on mobile?

■ Why Interviewers Ask This

Tests resource awareness.

■ Strong Candidate Answer

Three big drains in a voice AI app: **persistent WebSocket**, **continuous mic recording**, and **screen on**. Mitigations: use **VAD** to record only when there's speech, use a **low-power codec** like Opus, **close the WS** when idle and reopen on next interaction, batch network calls to wake the radio less often, and let the screen dim if the user isn't interacting. I profile battery with **Android Battery Historian** and **Xcode Energy Impact** — both show which subsystem is eating power. A poorly tuned voice AI app can drain a phone in 2 hours; a well-tuned one lasts 6–8.

■ Pro Tip

*Mention the **2-hour vs 8-hour** contrast — concrete numbers make the case memorable.*

QUESTION #99

■ Q99. What is on-device AI, and which frameworks would you use — Core ML, TFLite, or ONNX?

■ Why Interviewers Ask This

Tests modern AI deployment knowledge.

■ Strong Candidate Answer

On-device AI means running the model directly on the phone, with no cloud call. Three main frameworks. **Core ML** on iOS — Apple's native framework, ships **.mlmodel** files, accelerated by the Neural Engine. **TFLite** — Google's, cross-platform, optimized for mobile, supports **GPU and NNAPI** acceleration on Android. **ONNX Runtime** — cross-platform, supports models converted from PyTorch, TensorFlow, and many others. In React Native I'd use **react-native-fast-tflite** for vision tasks, or **react-native-vision-camera** for real-time camera + ML.

■ Pro Tip

*Name **react-native-fast-tflite** by name — it's the modern RN-friendly choice and interviewers notice when you know specific libraries.*

QUESTION #100

■ Q100. When should you pick on-device AI vs cloud AI?

■ Why Interviewers Ask This

Tests product-engineering judgment.

■ Strong Candidate Answer

On-device wins when you need **privacy** (data never leaves the phone), **offline capability**, **no per-request cost**, or **ultra-low latency**. Cloud wins when you need **large, state-of-the-art models** (LLMs are too big for phones), **centralized updates**, or **compute-heavy** tasks. Most production apps end up **hybrid**: on-device for small classifiers, wake words, image filters, vision tasks; cloud for large LLMs and anything that needs broader knowledge. A useful rule: if the model is over **2GB** compressed, it probably belongs in the cloud — most phones can't load it without killing other apps.

■ Pro Tip

*Mention **'hybrid is the default'** — pure on-device or pure cloud is rare in production.*

Good luck, Rinshad.

Confidence comes from preparation. You've already done the work — this guide just helps you say it out loud.

— Prepared with care, 2026